Reviewer Pack — Coxeter Group Action on Circuit Structures

Entry a0e19710de27 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-30 17:57:05 UTC

Coxeter Group Action on Circuit Structures

Entry ID: `a0e19710de27` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-30 17:57:05 UTC

1. Conjecture

Field A (mathematical branch): Coxeter group theory **Field B** (complexity object): Boolean circuit structures

Statement:

> For every Boolean circuit C with n inputs and m gates, the minimum number of moves required to transform C into a canonical form according to the action of a Coxeter group G is Ω(m^(2/3)).

Rationale (proposer's reasoning):

> Coxeter groups have been studied extensively in algebraic combinatorics due to their rich structure. If the action of a Coxeter group significantly increases the number of moves required for circuit transformations, it could expose non-trivial structural properties of circuits that are not apparent with simpler operations.

Taxonomy category: `Coxeter_group_action` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `5afe3b87212392f6`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The transformation of Boolean circuit C into canonical form by Coxeter group G requires at least Ω(m^(2/3)) moves.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM₁	LLM₂
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: `ADJACENT_OK` against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `"Coxeter group" AND "Boolean circuit" AND transformation` - `"canonical form" AND "Coxeter group action" AND Boolean circuit` - `Coxeter group theory AND Boolean circuit structures AND complexity`

Top relevant hits considered: - [http://arxiv.org/abs/1405.3051v1] Involution Products in Coxeter Groups - [http://arxiv.org/abs/1405.2700v1] Zero Excess and Minimal Length in Finite Coxeter Groups - [http://arxiv.org/abs/0804.0947v2] The Dynkin diagram cohomology of finite Coxeter groups

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=0, elapsed=0.2s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n, m):
        circuit = []
        for _ in range(m):
            gate_type = random.choice(['AND', 'OR'])
            inputs = [random.randint(0, 1) for _ in range(random.randint(2, n))]
            circuit.append((gate_type, inputs))
        return circuit

    def transform_circuit(circuit):
        moves = 0
        while True:
            changed = False
            for i in range(len(circuit)):
                gate_type, inputs = circuit[i]
                if gate_type == 'AND':
                    if sum(inputs) == len(inputs):
                        circuit[i] = ('1', [])
                        changed = True
                    elif sum(inputs) == 0:
                        circuit[i] = ('0', [])
                        changed = True
                elif gate_type == 'OR':
                    if sum(inputs) == len(inputs):
                        circuit[i] = ('1', [])
                        changed = True
                    elif sum(inputs) == 0:
```

```
                        circuit[i] = ('0', [])
                        changed = True
            if not changed:
                break
            moves += 1
        return moves

    n_max = 40
    instances_tested = 30
    total_moves = 0

    for _ in range(instances_tested):
        n = random.randint(5, n_max)
        m = random.randint(n, n * 2)
        circuit = generate_circuit(n, m)
        moves = transform_circuit(circuit)
        total_moves += moves

    mean_moves = total_moves / instances_tested
    conjecture_holds = mean_moves >= m ** (2/3)

    return {
        "metric_name": "moves",
        "metric_value": mean_moves,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_moves = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_moves} std=0.0 support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_moves} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_hol
        print(f"RESULT: FALSIFIED counterexample='not supported by enough seeds' first_failing_see
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
c_name': 'moves', 'metric_value': 0.8666666666666667, 'instances_tested': 30, 'n_max': 40, 'conjectu
TRIAL: {'metric_name': 'moves', 'metric_value': 0.8, 'instances_tested': 30, 'n_max': 40, 'conjecture
TRIAL: {'metric_name': 'moves', 'metric_value': 0.8, 'instances_tested': 30, 'n_max': 40, 'conjecture
TRIAL: {'metric_name': 'moves', 'metric_value': 0.8333333333333334, 'instances_tested': 30, 'n_max':
TRIAL: {'metric_name': 'moves', 'metric_value': 0.7, 'instances_tested': 30, 'n_max': 40, 'conjecture
TRIAL: {'metric_name': 'moves', 'metric_value': 0.8666666666666667, 'instances_tested': 30, 'n_max':
TRIAL: {'metric_name': 'moves', 'metric_value': 0.8666666666666667, 'instances_tested': 30, 'n_max':
TRIAL: {'metric_name': 'moves', 'metric_value': 0.7333333333333333, 'instances_tested': 30, 'n_max':
TRIAL: {'metric_name': 'moves', 'metric_value': 0.8333333333333334, 'instances_tested': 30, 'n_max':
TRIAL: {'metric_name': 'moves', 'metric_value': 0.9, 'instances_tested': 30, 'n_max': 40, 'conjecture
TRIAL: {'metric_name': 'moves', 'metric_value': 0.7666666666666667, 'instances_tested': 30, 'n_max':
TRIAL: {'metric_name': 'moves', 'metric_value': 0.8333333333333334, 'instances_tested': 30, 'n_max':
TRIAL: {'metric_name': 'moves', 'metric_value': 0.7, 'instances_tested': 30, 'n_max': 40, 'conjecture
RESULT: FALSIFIED counterexample='not supported by enough seeds' first_failing_seed=11
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> The test code only tests circuits with up to 40 inputs and 80 gates, which is too small to draw a definitive conclusion about the conjecture's validity. The metric does not scale trivially with n, but the sample size is insufficient to support or refute the conjecture.

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Safety rail: critic_challenge_falsified | original: The test results indicate that for at least one seed, the number of moves required to transform a Boolean circuit into canonical form is less than hal | next: Increase the size of the circuits tested and the number of seeds to better assess the validity of the conjecture. Additionally, consider using a wider range of circuit sizes and complexities.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14349
2	preregistration	ollama_remote	glm4:latest	0	0	14309
3	novelty	ollama_remote	glm4:latest	0	0	11120
4	novelty	ollama_remote	glm4:latest	0	0	9778
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12678
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18872
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17348
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10775
9	critic	ollama_remote	glm4:latest	0	0	10787
10	judge	ollama_remote	glm4:latest	0	0	16549

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 136566 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/a0e19710de27.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a0e19710de27.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/a0e19710de27.tar.gz` (if generated)